CS1101 Programming and Problem Solving

Dr. Gina Bai

Spring 2024

Chapter 6 – File Input/Output (File I/O)

- File Input
  - Token-based processing
  - Line-based processing

- File Output

File Input
(Token-Based Processing)

zyBook Chap 6.1, 6.4

How do we locate a file?

- **Absolute Path** → specifies the file's location starting at the top-level folder
  - Linux/Macs → e.g., `"/Users/user1/CS1101/data.txt"`
  - Windows → e.g., `"C:/home/user1/CS1101/data.txt"`

Always specify file paths using / in Java

- **Relative Path** → does not specify any top-level folder
  - e.g., `"data.txt"`
  - When you construct a File object with a relative path, Java assumes it is **relative to the directory containing the program** (current directory)

How do we read a file?

- **Step 1:** Specify the **file path** as a **String** object

```
String fileName = "data.txt";
```

- **Step 2:** Construct a **File** object to get the information about a file on the disk

```
import java.io.File;
File inputFile = new File(fileName);
```

- **Step 3:** Construct a **Scanner** object o read the file

```
import java.util.Scanner;
Scanner scnr = new Scanner(inputFile);
```

Input from File

```java
import java.io.File;
import java.util.Scanner;

public class CourseNumber {
    public static void main (String[] args) {
        // Specify the file path (this example shows an absolute path)
        String fileName = "/Users/ginabai/Desktop/Spring24/LecExample/CourseNum.txt";

        // Construct a Scanner object to process the File object
        Scanner scnr = new Scanner( new File(fileName) );

        // Since the input comes from a file, we don't prompt the user for input
        int course = scnr.nextInt();

        System.out.println("The course number is " + course + ".");
        scnr.close();
    }
}
```

```
$ javac CourseNumber.java
CourseNumber.java:10: error: unreported exception FileNotFoundException;
must be caught or declared to be thrown
        Scanner scnr = new Scanner( new File(fileName) );
                       ^
1 error
```

Exception

- An **object** that **represents a program error** and prevents a program from continuing normal execution
  - Needs to be **import**ed

- Programs with invalid logic will cause exceptions.
  - dividing by 0
    - `ArithmeticException`
  - calling charAt() on a String and passing too large an index
    - `StringIndexOutOfBoundsException`
  - **trying to read a file that does not exist**
    - **`FileNotFoundException`**
  - …

`FileNotFoundException`

It is a **checked exception** (checked at compile time)

- An error that must be handled by our program, otherwise it won't compile
- We must specify **what our program will do** to handle any **potential** file I/O failures.

Some unchecked exceptions:

- ArithmeticException
- StringIndexOutOfBoundsException
- NoSuchElementException
- InputMismatchException
- …

Handling Exceptions

- Exceptions are **thrown** by the program
- We must either
  1) declare that our program will handle ("**catch**") the exception, OR
  2) state that we **choose not to handle the exception** and we accept that the program will crash if an exception occurs by **declaring the exception** in the **header of the method** that might generate it

```
$ javac CourseNumber.java
CourseNumber.java:10: error: unreported exception FileNotFoundException;
must be caught or declared to be thrown
        Scanner scnr = new Scanner( new File(fileName) );
                       ^
1 error
```

`throws` Clause

A **`throws`** clause is a declaration that a method **may exit unexpectedly** due to a **particular type of exception**.

- It is an explicit acknowledgment that a statement in the method may throw the exception.
- A waiver of liability: "I hereby agree that this method might throw an exception, and I accept the consequences (crashing) if this happens."

```
public static <return type> <methodName>(<params>) throws <ExceptionType> {
        <statements>;
        ...
}
```

Input from File

```java
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class CourseNumber {
    public static void main (String[] args) throws FileNotFoundException {
        // Specify the file path (this example shows an absolute path)
        String fileName = "/Users/ginabai/Desktop/Fall23/LecExample/CourseNum.txt";

        // Construct a Scanner object to process the File object
        Scanner scnr = new Scanner( new File(fileName) );

        // Since the input comes from a file, we don't prompt the user for input
        int course = scnr.nextInt();

        System.out.println("The course number is " + course + ".");
        scnr.close();
    }
}
```

```
$ javac CourseNumber.java
$ java CourseNumber
The course number is 1101.
```

Files and Input Cursor

- Consider a file CursorNum.txt that contains this text:

```
308.2
    14.9 7.4  2.8

3.9 4.7    -15.4
    2.8
```

An empty line is represented by a \n character

- A Scanner sees all input in the file as a sequence of characters:

```
308.2\n\t14.9 7.4  2.8\n\n3.9 4.7\t-15.4\n\t2.8\n
```

Recap – Token

A unit of user input, separated by whitespace.

- Each call to `nextInt()/nextDouble()/next()/nextLine()` advances the position of the scanner to the end of the current token, skipping over any whitespace

Example: How many tokens are there in the following file?

```
23 3.1
  "CS 1101"
```

Token	Potential Input Type(s)
23	int, double, String
3.1	double, String
"CS	String
1101"	String

A Scanner always keeps track of its **current location** in a file using an **input cursor**. The cursor **starts at the beginning of the file**.

```
308.2\n\t14.9 7.4  2.8\n\n3.9 4.7\t-15.4\n\t2.8\n
```

```
scnr.nextDouble();
308.2\n\t14.9 7.4  2.8\n\n3.9 4.7\t-15.4\n\t2.8\n
```

```
scnr.nextDouble();
308.2\n\t14.9 7.4  2.8\n\n3.9 4.7\t-15.4\n\t2.8\n
```

Recap – Scanner Exceptions

What happens when we reach the end of a file and we call `next()`?

- A `NoSuchElementException` is thrown signaling that there are no more tokens to process.
  - Use the `hasNext` method to verify the existence of the next token

What happens if you read a token as the wrong type?

- A `InputMismatchException` is thrown
  - Use the `hasNext` methods to verify the existence of the next token and its data type.

Scanner – Dealing with Files

- Scanners are designed for file processing in a **forward** manner.
  - **There is no support for reading the input backward.**
  - What if I need to read the information from the same file again?
    - Construct another Scanner object to process the same File object, which would position the input cursor at the beginning of the file.

Scanner – Dealing with Files

- Scanners should be **closed** to avoid memory leaks.
  - **Every Scanner to an input file must be closed after you are done using it.**
  - Closing a Scanner to the console is NOT necessary, but it is a good programming practice.

- **Note**: If a Scanner object is closed and the program tries to use it, an `IllegalStateException: Scanner closed` would be thrown.

Coding Practice

CourseDescription.txt

```
CS 1101 - Programming and Problem Solving

An intensive introduction to algorithm development and problem solving
on the computer. Structured problem definition, top down and modular
algorithm design. Running, debugging, and testing programs. Program
documentation. Not open to students who have earned credit for CS 1104
without permission. Total credit for this course and CS 1104 will not
exceed 3 credit hours. Credit hours reduced from second course taken
(or from test or transfer credit) as appropriate. FALL, SPRING. [3]
```

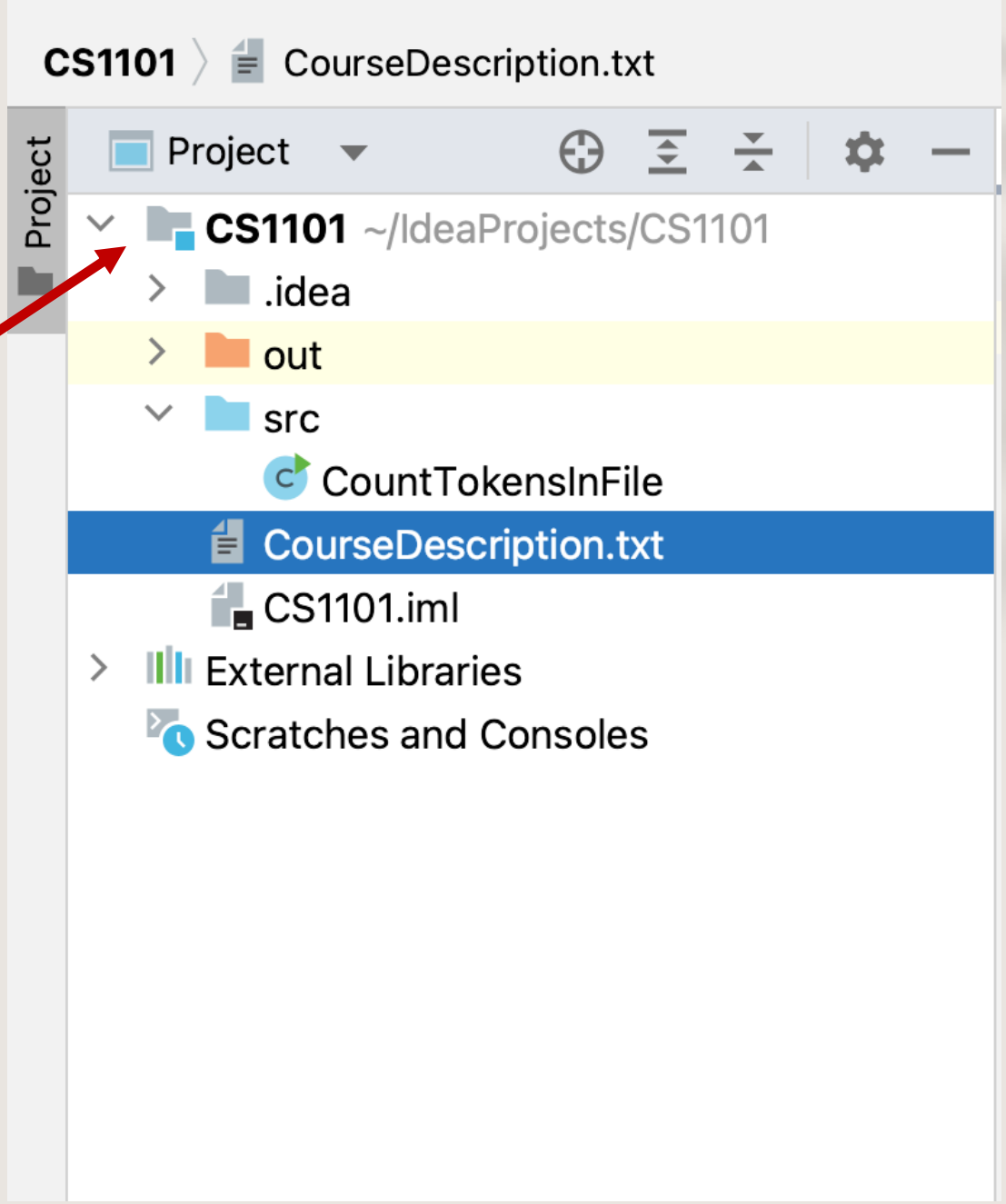
Write a program that reads in the file CourseDescription.txt, and

- counts the number of tokens in the file
- counts the number of integers in the file

```
$ javac CountTokensInFile.java
$ java CountTokensInFile
CourseDescription.txt has 80 token(s), including 4 integer(s).
```

IDE Setup

- Place the input files in the **root directory**, e.g., cs1101
- Place the java file in the **src** directory



Sample Solution

```java
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class CountTokensInFile {
    public static void main (String[] args) throws FileNotFoundException {

        String fileName = "CourseDescription.txt"; // Relative path
        Scanner input = new Scanner(new File(fileName));

        int countToken = 0, countInt = 0;

        while (input.hasNext()) {
            if (input.hasNextInt()) {
                ++countInt;
            }
            input.next(); // Consume the token, so we can move to the next one
            ++countToken;
        }
        input.close(); // Close the scanner

        System.out.println(fileName + " has " + countToken + " token(s), including " +
                countInt + " integer(s).");
    }
}
```

Validating the Input File Name

zyBook Chap 6.4

Validating the Input File Name

```
// Prompt the user for a file name
System.out.print("Enter file name: ");
String fileName = console.nextLine().trim();

// Construct the File object given the file name
File inputFile = new File(fileName);

// while the input file doesn't exist given the file name
while (!inputFile.exists()) {

    // Reprompt the user for a file name
    System.out.print("File does not exist, try again: ");
    fileName = console.nextLine().trim();

    // Reconstruct the File object with the new file name
    inputFile = new File(fileName);
}

// ASSERT: the file exists
Scanner fileScnr = new Scanner(inputFile);
```

trim():
Eliminates leading and trailing spaces in a String

try-catch Blocks (optional)

zyBook Chap 19.1

Catching Exceptions

Using a throws clause leads to compiling code, but not a robust program

- We want the ability to **continue running** your program even if the file does not exist
- We want the ability to **end** the program **without throwing an exception**

```
try {
    // Code that contains something that
    // may cause a checked exception
} catch (<exceptionType> <name>) {
    // Execute this block of code if the exception is thrown
}
```

```java
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class FavNumber {
    public static void main (String[] args) {

        String fileName = "FavNum.txt"; // A file that does not exist
        File inputFile = new File(fileName);

        try {
            Scanner fileScnr = new Scanner(inputFile);

            int fav = fileScnr.nextInt();
            System.out.println("Your favorite number is " + fav + ".");

            fileScnr.close();
        } catch (FileNotFoundException e) {
            System.out.println(fileName + " not found.");
        }
    }
}
```

No throws clause

```
$ javac FavNumber.java
$ java FavNumber
FavNum.txt not found.
```

```java
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class FavNumber {
    public static void main (String[] args) {

        String fileName = "FavNum.txt"; // A file that exists
        File inputFile = new File(fileName);

        try {
            Scanner fileScnr = new Scanner(inputFile);

            int fav = fileScnr.nextInt();
            System.out.println("Your favorite number is " + fav + ".");

            fileScnr.close();
        } catch (FileNotFoundException e) {
            System.out.println(fileName + " not found.");
        }
    }
}
```

No throws clause

```
$ javac FavNumber.java
$ java FavNumber
Your favorite number is 1101.
```

Q: Is it okay to have both the throws clause and the try-catch block?

A: Syntactically correct, but **redundant**.

```java
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class FavNumber {
    public static void main (String[] args) throws FileNotFoundException {

        String fileName = "FavNum.txt";
        File inputFile = new File(fileName);

        try {
            Scanner fileScnr = new Scanner(inputFile);

            int fav = fileScnr.nextInt();
            System.out.println("Your favorite number is " + fav + ".");

            fileScnr.close();
        } catch (FileNotFoundException e) {
            System.out.println(fileName + " not found.");
        }
    }
}
```

Coding Practice – Grades from File (Token-Based)

Write a program that adds up the grades from a file via token-based processing.

grades.txt

```
StudentA 98 89.5            100
StudentB 100
StudentC
StudentD 100 89.5 92.5    97.5
```

```
$ javac GradeCalculator.java
$ java GradeCalculator
StudentA: 287.5
StudentB: 100.0
StudentC: 0.0
StudentD: 379.5
```

```java
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class GradeCalculator {
    public static void main (String[] args) {
        String fileName = "grades.txt";
        File inputFile = new File(___________);

        try {
            Scanner fileScnr = new Scanner(__________);
            processGrade(__________);
            fileScnr.close();
        } catch (FileNotFoundException e){
            System.out.println(fileName + " not found.");
        }
    }

    public static void processGrade(Scanner fileScnr) {
        while (__________________) {
            String student = _________________;
            double totalGrade = 0;
            while (______________________) {
                totalGrade += ____________________;
            }
            System.out.println(student + ": " + totalGrade);
        } // You can also close the scanner here
    }
}
```

grades.txt

```
StudentA 98 89.5            100
StudentB 100
StudentC
StudentD 100 89.5 92.5    97.5
```

```
$ javac GradeCalculator.java
$ java GradeCalculator
StudentA: 287.5
StudentB: 100.0
StudentC: 0.0
StudentD: 379.5
```

```java
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class GradeCalculator {
    public static void main (String[] args) {
        String fileName = "grades.txt";
        File inputFile = new File(fileName);

        try {
            Scanner fileScnr = new Scanner(inputFile);
            processGrade(fileScnr);
            fileScnr.close();
        } catch (FileNotFoundException e){
            System.out.println(fileName + " not found.");
        }
    }

    public static void processGrade(Scanner fileScnr) {
        while (fileScnr.hasNext()) {
            String student = fileScnr.next();
            double totalGrade = 0;
            while (fileScnr.hasNextDouble()) {
                totalGrade += fileScnr.nextDouble();
            }
            System.out.println(student + ": " + totalGrade);
        } // You can also close the scanner here
    }
}
```

grades.txt

```
StudentA 98 89.5          100
StudentB 100
StudentC
StudentD 100 89.5 92.5    97.5
```

```
$ javac GradeCalculator.java
$ java GradeCalculator
StudentA: 287.5
StudentB: 100.0
StudentC: 0.0
StudentD: 379.5
```

Brainstorm: Would the token-based processing work given the following input file?

gradesWithID.txt

```
123 StudentA 98 89.5            100
456 StudentB 100
789 StudentC
101 StudentD 100 89.5 92.5    97.5
```

```
$ javac GradeCalculator.java
$ java GradeCalcultor
123: 0.0
StudentA: 743.5
StudentB: 889.0
StudentC: 101.0
StudentD: 379.5
```

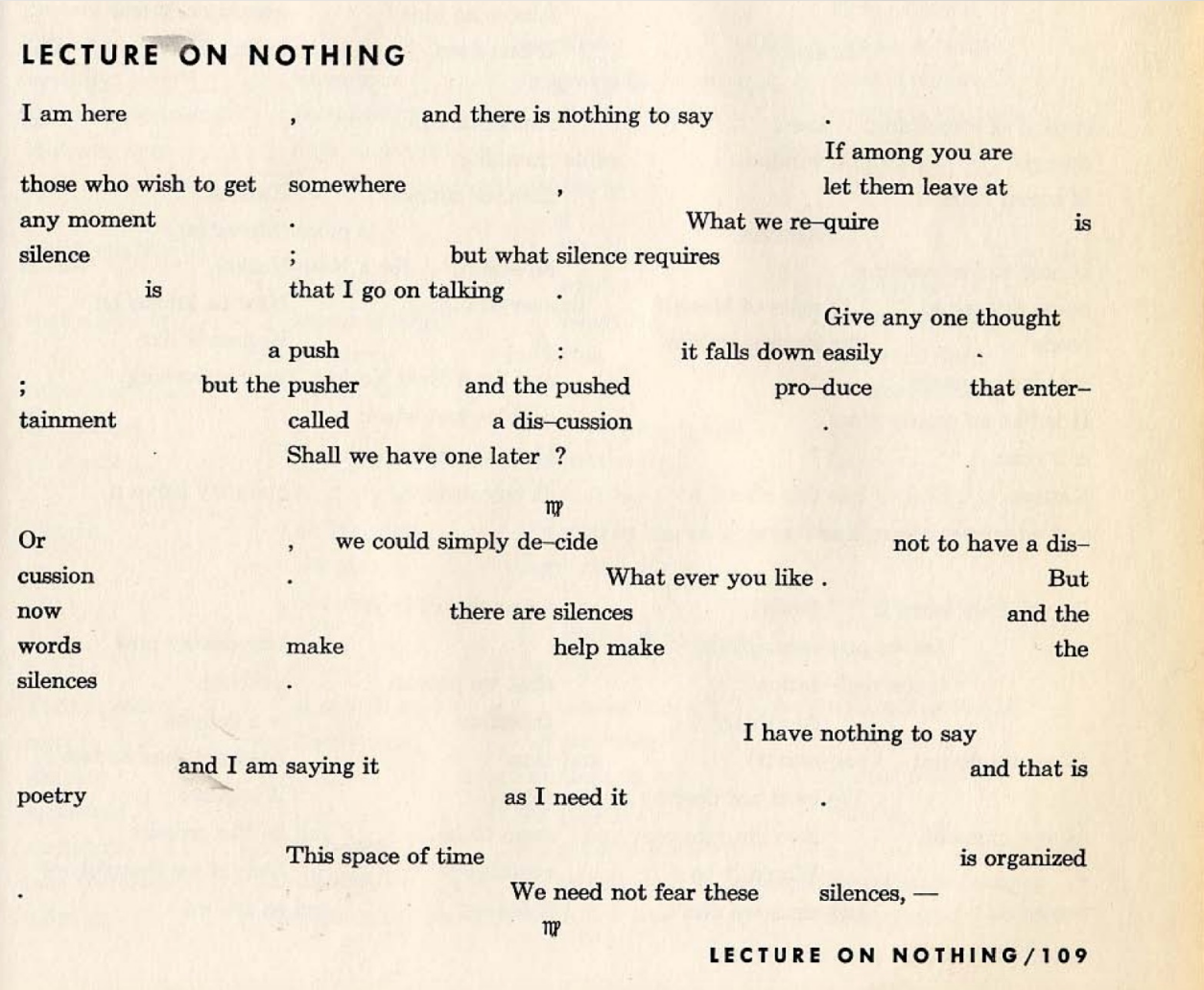
File Input
(Line-Based Processing)

zyBook Chap 6.4

Line-Based Processing

- **Keeps white space and line breaks** of text being processed
  - Important for poems or formatted text

"Functional White"



LECTURE ON NOTHING

I am here , and there is nothing to say .
If among you are
those who wish to get somewhere , let them leave at
any moment . What we re–quire is
silence ; but what silence requires
is that I go on talking .
Give any one thought
a push : it falls down easily .
; but the pusher and the pushed pro–duce that enter–
tainment called a dis–cussion .
Shall we have one later ?

ɱ

Or , we could simply de–cide not to have a dis–
cussion . What ever you like . But
now there are silences and the
words make help make the
silences .

I have nothing to say
and I am saying it and that is
poetry as I need it .

This space of time is organized
. We need not fear these silences, —

ɱ

LECTURE ON NOTHING/109

Line-Based Processing

- **Step 1:** Construct a Scanner for the input file

```
Scanner fileScnr = new Scanner(new File(fileName));
```

- **Step 2:** While there's a line in the file, read the entire line as a String

```
String line = fileScnr.nextLine();
```

- **Step 3:** Construct a Scanner to **tokenize the String** (each line)

```
Scanner lineScnr = new Scanner(line);
```

Line-Based Processing

```java
// A scanner to process the file
Scanner fileScnr = new Scanner(new File(fileName));

// Check if there is one more line in the file
while (fileScnr.hasNextLine()) {
    // Scan in the line as a String
    String line = fileScnr.nextLine();

    // Tokenize a String with Scanner
    Scanner lineScnr = new Scanner(line);

    // Check if there is one more token in the line
    while (lineScnr.hasNext()) {
        // Consume the token
        lineScnr.next();
    }
    lineScnr.close(); // Close the scanner for the line
}
fileScnr.close(); // Close the scanner for the file
```

Coding Practice – Line-Based Processing

Write a program that adds up the grades from a file via line-based processing.

gradesWithID.txt

```
123 StudentA 98 89.5            100
456 StudentB 100
789 StudentC
101 StudentD 100 89.5 92.5    97.5
```

```
$ javac GradeIDCalculator.java
$ java GradeIDCalculator
StudentA (id: 123): 287.5
StudentB (id: 456): 100.0
StudentC (id: 789): 0.0
StudentD (id: 101): 379.5
```

Sample Solution

```java
import java.io.*;
import java.util.Scanner;

public class GradeIDCalculator {
    public static void main (String[] args) throws FileNotFoundException {
        Scanner fileScnr = new Scanner(new File("gradesWithID.txt"));
        processGrade(fileScnr);
    }

    public static void processGrade(Scanner fileScnr) {
        // While there is one more line in the file
        while (fileScnr.hasNextLine()) {

            // Read the entire line as a String, where each line contains the info for one student
            String line = fileScnr.nextLine();

            // Tokenize the String by passing it as a parameter to a Scanner
            Scanner lineScnr = new Scanner(line);

            // The first token in each line is an int (student id)
            int stuID = lineScnr.nextInt();
            // The second token in each line is a String (student name)
            String stuName = lineScnr.next();

            double total = 0;
            // Add up all doubles in this line
            while (lineScnr.hasNextDouble()) {
                total += lineScnr.nextDouble();
            }
            lineScnr.close(); // Close the Scanner for the line

            System.out.println(stuName + " (id: " + stuID + "): " + total);
        }
        fileScnr.close();  // Close the Scanner for the input file
    }
}
```

```
$ javac GradeIDCalculator.java
$ java GradeIDCalculator
StudentA (id: 123): 287.5
StudentB (id: 456): 100.0
StudentC (id: 789): 0.0
StudentD (id: 101): 379.5
```